

Carnet de pêche — Récapitulatif projet & guide d'itération

Document de référence à garder à la racine du dossier projet.

Dernière mise à jour : 11 juin 2026

1. Le projet en bref

Application web personnelle de suivi des sorties de pêche, couvrant la **chasse sous-marine (spearfishing)** et la **pêche à la canne classique**.

Décisions clés :

- **Stack** : application web légère (HTML / CSS / JS vanilla), carte via Leaflet.js + tuiles OpenStreetMap (gratuit, sans clé API).
- **Stockage** : 100 % local (localStorage du navigateur). Pas de backend pour l'instant.
- **Profil** : multi-utilisateurs en local (sélecteur de profil), données isolées par profil. Structure pensée pour brancher plus tard un vrai backend (Supabase) si besoin de comptes synchronisés multi-appareils.
- **Photos** : stockées en local (base64), compressées avant stockage pour ne pas saturer le localStorage (~5-10 Mo max).
- **Sauvegarde** : boutons Export / Import JSON pour ne pas perdre les données (le localStorage peut être effacé).

Fonctionnalités, par ordre de priorité :

1. **Carte interactive** des spots (mer / eau douce, notes par spot, ajout au clic).
 2. **Logger de session** complet (type de pêche, conditions météo/mer/marée, profondeur, technique/appât, captures avec espèce/taille/poids/photo, notes).
 3. **Planificateur** « quand y aller ? » (conditions + historique de succès ; bonus météo via Open-Meteo, sans clé).
 4. **Statistiques & achievements** (totaux, plus grosse prise, espèce la plus fréquente, badges, graphiques).
-

2. Contexte abonnement & modèles (plan Pro)

- Sur **Pro**, pas de coût au token : tout est inclus dans l'abonnement.
- L'usage est un **pool partagé entre le chat Claude et Claude Code** : fenêtre glissante de 5 h + limite hebdomadaire.
- **Opus 4.8** consomme le budget beaucoup plus vite que **Sonnet 4.6**.
- **Reco modèle** : faire le gros du travail en **Sonnet 4.6** (largement suffisant pour du JS vanilla), passer en **Opus 4.8** uniquement pour les points d'architecture épineux.

- Suivre la conso avec `/usage` ou `/status` dans Claude Code.
 - Garder le client **Claude Code à jour** (un bug de sur-consommation Opus 4.8 a été corrigé le 1er juin 2026).
-

3. Best practices pour faire évoluer le site

Filet de sécurité (le plus important)

- **Git** : initialiser le dépôt et **commit après chaque version qui marche**. En cas de casse, on revient en arrière en une commande.
 - Au besoin : « commit l'état actuel avant qu'on continue ».
- À défaut : copier le dossier complet avant une grosse modif.

Itérer proprement

- **Une modification à la fois**, puis tester dans le navigateur avant la suivante. Plus sûr et moins gourmand en budget.
- **Être précis** : désigner l'élément, décrire le comportement attendu, donner un exemple.
- **Débogage** : coller l'erreur exacte (console du navigateur, touche F12) plutôt que « ça marche pas ».

Garder le cap entre les sessions

- Créer un fichier `CLAUDE.md` à la racine, relu à chaque session (stack, structure des données, principe « simplicité et robustesse »).
 - « crée un CLAUDE.md qui documente ce projet ».
- Démarrer une **session fraîche** (ou `/compact`) pour une tâche sans rapport avec la précédente.

Spécifique à cette appli

- Toute modif de la **structure des données** peut casser/effacer les données existantes.
 - Avant : **exporter le JSON** (bouton prévu).
 - Demander : « gère la migration des données existantes sans les perdre ».

Pour les changements ambitieux

- Demander de **planifier avant de coder** : « ne code pas encore, explique-moi d'abord comment tu t'y prendrais ». Valider l'approche, puis laisser faire.
-

4. Prochaine étape recommandée

Sécuriser et tester la boucle de base avant d'ajouter de nouvelles fonctionnalités.

1. **Sécuriser** : faire initialiser Git + un premier commit de la version actuelle, et générer le `CLAUDE.md`.
2. **Tester pour de vrai** : ajouter un spot réel (ex. Port-Blanc), logger une vraie session avec photo, vérifier que tout se sauvegarde et se recharge après fermeture du navigateur.
3. **Noter les frictions** : faire une liste de ce qui est cassé, manquant ou pas pratique.
4. **Itérer** sur cette liste (une modif à la fois), puis seulement ensuite avancer sur le planificateur, puis les stats.

■ Principe : on stabilise et on rend agréable le cœur (carte + logger) avant d'élargir.